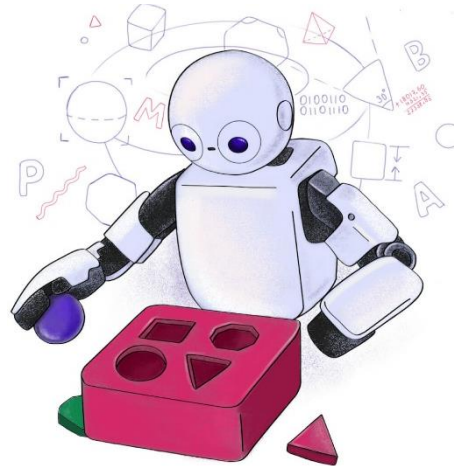
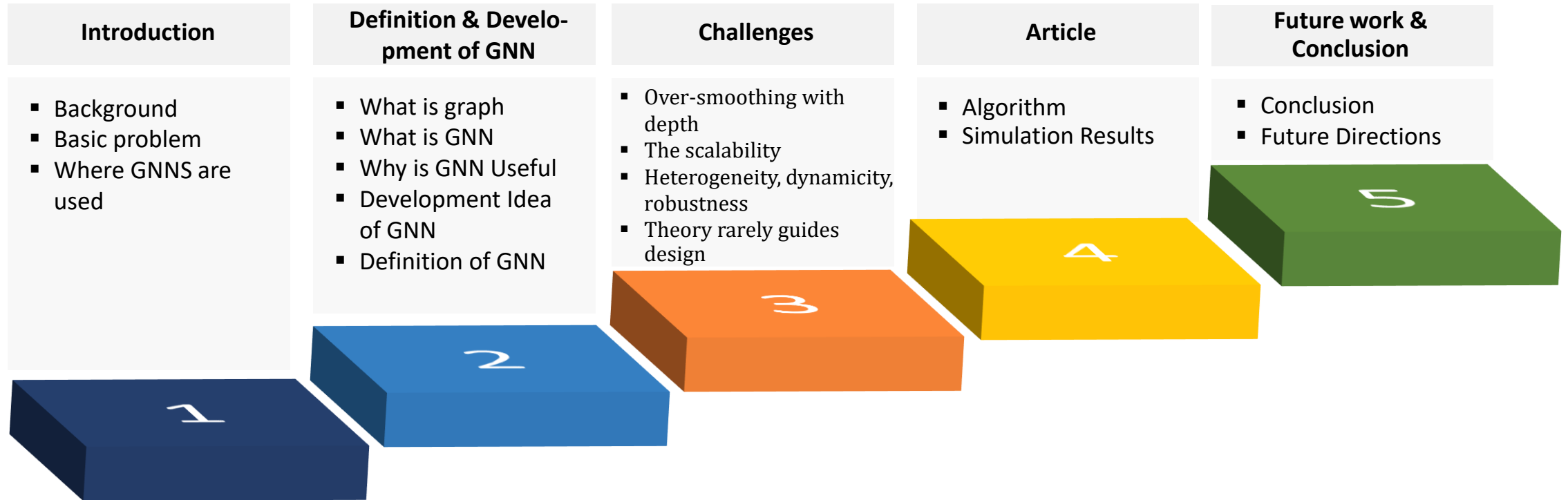


TP558 - Tópicos avançados em
Machine Learning

Graph Neural Networks



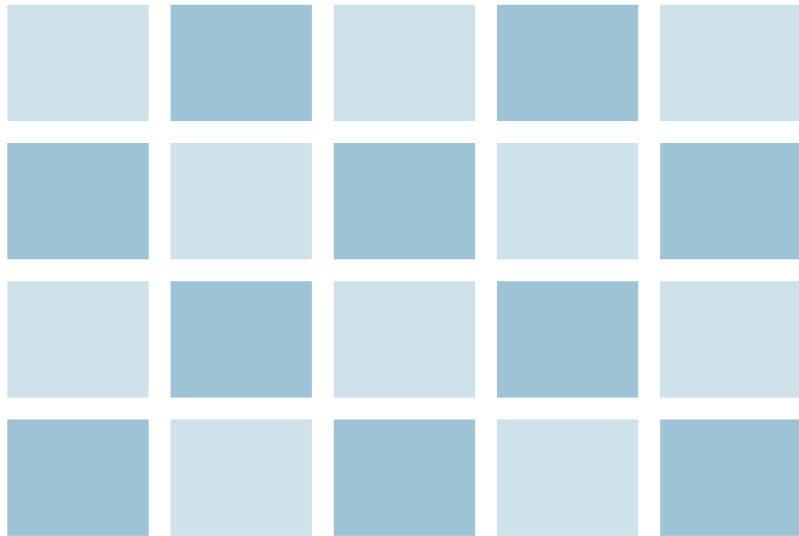
Outline



Introduction

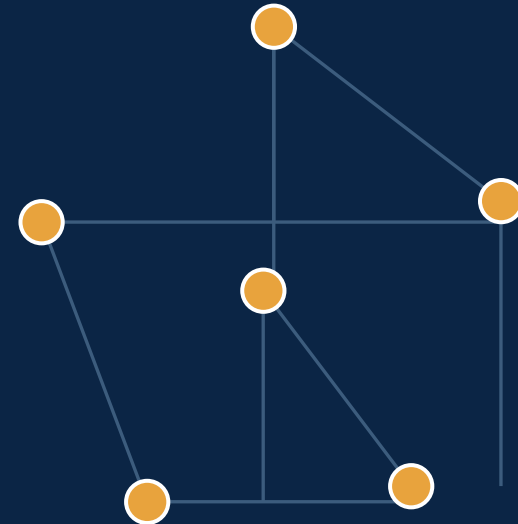
Most real-world data is not a grid

Euclidean data (images, text)



Fixed neighborhoods, ordered pixels/tokens — CNNs and RNNs exploit shift-invariance and local connectivity.

Non-Euclidean data (graphs)



Variable-size, unordered neighborhoods: molecules, social networks, citation graphs, sensor and wireless networks.

Zhou et al., “Graph Neural Networks: A Review of Methods and Applications,” AI Open, 2020; Wu et al., “A Comprehensive Survey on Graph Neural Networks,” 2019/2021.

Why standard neural networks fall short on graphs

1

No fixed node ordering

A graph has no canonical order for its nodes — CNN filters and RNN sequences assume one.

2

Variable neighborhood size

Each node can have a different number of neighbors; convolution kernels expect a fixed receptive field.

3

Broken independence assumption

Traditional ML assumes i.i.d. samples; graph nodes are explicitly linked by edges of many types.

4

Structure carries information

Flattening a graph into a feature vector during preprocessing discards topology that is often the signal itself.

Zhou et al., “Graph Neural Networks: A Review of Methods and Applications,” AI Open, 2020; Wu et al., “A Comprehensive Survey on Graph Neural Networks,” 2019/2021.

GNNs already power a wide range of domains



Chemistry & Biology

Molecular fingerprints, drug discovery, protein interface prediction



Physics

Particle & robotic systems, interaction/relational reasoning



Knowledge Graphs

Entity embedding, link prediction, KG alignment



Computer Vision

Scene graphs, point clouds, few-shot & zero-shot recognition



Text & NLP

Text classification, relation & event extraction, NMT



Recommendation

User-item graphs, social recommendation (PinSAGE, GC-MC)



Traffic Networks

Spatial-temporal forecasting of flow, speed, demand



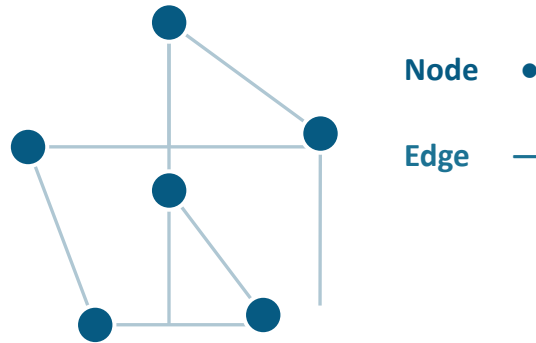
Wireless Networks

Resource management, beamforming, network modeling — today's case study

Zhou et al., "Graph Neural Networks: A Review of Methods and Applications," AI Open, 2020; Wu et al., "A Comprehensive Survey on Graph Neural Networks," 2019/2021.

Definition and Development of GNN

Formal notation: graphs, nodes, and neighborhoods



- $G = (V, E)$: set of nodes V , set of edges E , $|V| = N$ nodes.
- Adjacency matrix $A \in \mathbb{R}^{N \times N}$ encodes connectivity.
- Node features $X \in \mathbb{R}^{N \times d}$; edges may carry features too.
- Neighborhood $N(v)$: the set of nodes connected to v .

Graph types

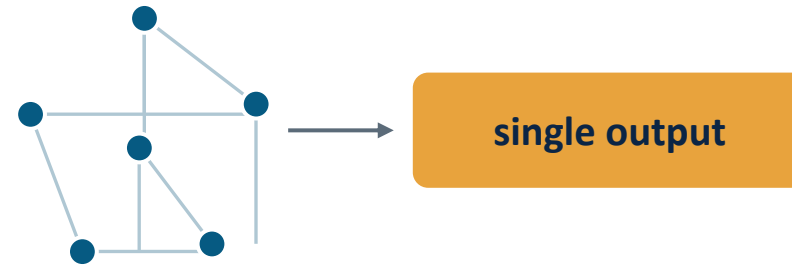
- Directed vs. undirected
- Homogeneous vs. heterogeneous (typed nodes/edges)
- Static vs. dynamic (topology/features vary in time)
- Positional vs. non-positional (ordered neighbor slots or not)

Zhou et al., "Graph Neural Networks: A Review of Methods and Applications," AI Open, 2020; Wu et al., "A Comprehensive Survey on Graph Neural Networks," 2019/2021.

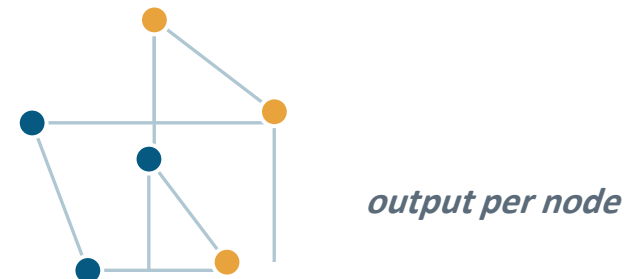
A GNN maps a graph (and a node) to a vector

- A Graph Neural Network learns a function $\tau(\mathbf{G}, \mathbf{n}) \in \mathbf{R}^m$ that maps a graph G and one of its nodes n to an output.
- Each node is given a state vector built from its own features and its neighbors' states — the state is learned, not hand-designed.
- **Two broad application classes**
 - Graph-focused: one output per whole graph (e.g., classify a molecule).
 - Node-focused: one output per node (e.g., label each router / user in a network).

Graph-focused



Node-focused

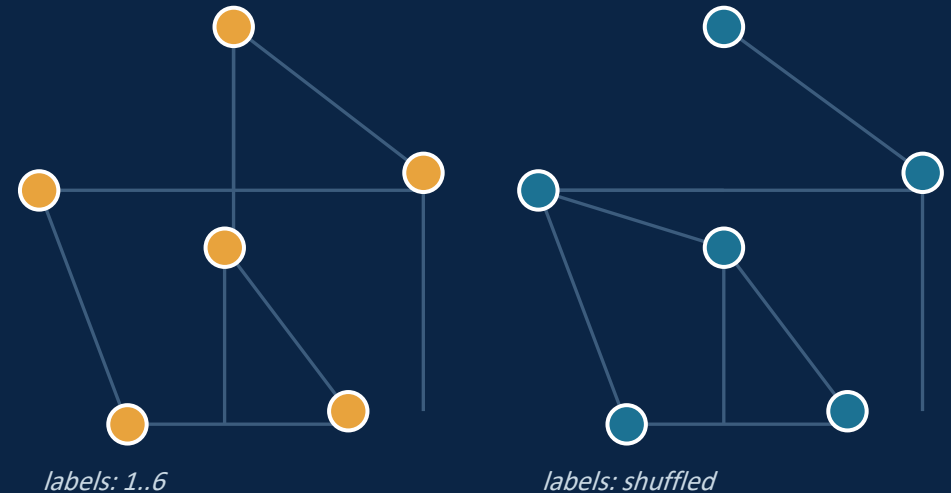


Zhou et al., "Graph Neural Networks: A Review of Methods and Applications," AI Open, 2020; Wu et al., "A Comprehensive Survey on Graph Neural Networks," 2019/2021.

Relational inductive bias & permutation invariance

- GNNs are equivariant to node/edge permutation: relabel the nodes and the model produces the same (reordered) answer.
- This “strong relational inductive bias” lets a GNN find symmetries and equivalent sub-structures between graphs seen in training and new graphs.
- Consequence: a GNN trained on one graph size/topology generalizes to unseen sizes and structures — fully-connected NNs cannot.

Same graph, different order → same model



→ identical output embedding, up to permutation

Zhou et al., “Graph Neural Networks: A Review of Methods and Applications,” AI Open, 2020; Wu et al., “A Comprehensive Survey on Graph Neural Networks,” 2019/2021.

Development idea: message passing

1 Initialize

Each node gets a hidden state from its own input features (zero-padded if needed).

2 Message

Every edge computes a message from the states of its two endpoints (+ edge features).

3 Aggregate

Each node combines incoming messages (sum / mean / max / attention) into one summary vector.

4 Update

The node combines the aggregated message with its previous state to get a new state.

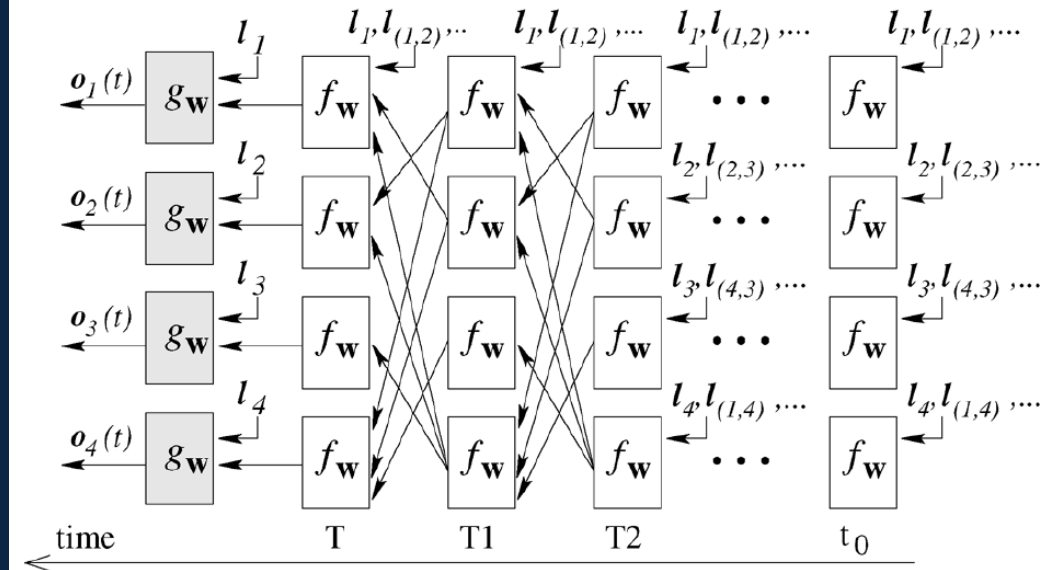
Repeated for T layers/iterations → information from T-hop neighborhoods reaches every node; run again on a completely different graph without retraining the same building blocks.

Zhou et al., "Graph Neural Networks: A Review of Methods and Applications," AI Open, 2020; Wu et al., "A Comprehensive Survey on Graph Neural Networks," 2019.

Scarselli et al. (2009): recurrent diffusion to a fixed point

- State equation: $\mathbf{x}_n = \mathbf{f}_w(\mathbf{l}_n, \mathbf{l}_{co}[n], \mathbf{x}_{ne}[n], \mathbf{l}_{ne}[n])$ — state of node n depends on its own label, edge labels, and neighbors' states.
- Output equation: $\mathbf{O}_n = \mathbf{g}_w(\mathbf{x}_n, \mathbf{l}_n)$
- States are updated iteratively (Jacobi-style) until they converge to a fixed point — guaranteed when $\mathbf{F}_w$ is a contraction mapping (Banach fixed-point theorem).
- Learning uses backpropagation-through-time on the “unfolded” encoding network (Almeida–Pineda algorithm).
- **This model can directly process cyclic, directed, and undirected graphs — unlike earlier recursive nets limited to DAGs.**

Encoding network (unfolded in time)



Each column = one time step, sharing the same weights f_w, g_w — a recurrent network unfolded over the graph.

Four families of modern GNNs

RecGNN

Recurrent GNNs

Propagate neighbor information until a stable equilibrium (original GNN, GraphESN, GGNN).

ConvGNN

Convolutional GNNs

Stack fixed layers, each with its own weights (spectral: ChebNet, GCN; spatial: GraphSAGE, GAT).

GAE

Graph Autoencoders

Encode nodes/graphs to a latent space and reconstruct structure — unsupervised embeddings and graph generation.

STGNN

Spatial-Temporal GNNs

Combine graph convolutions with RNNs/1D-CNNs to model graphs that evolve over time (e.g., traffic).

Zonghan Wu et al., "A Comprehensive Survey on Graph Neural Networks ," arXiv:1901.00596v4, 2019

Two ways to define “convolution” on a graph

Spectral approaches

- Define filters in the graph-Fourier (Laplacian eigenvector) domain.
- ChebNet, GCN approximate the filter with polynomials to avoid full eigendecomposition.
- Limitation: filters are tied to a fixed graph structure — harder to transfer across graphs.

Spatial approaches

- Aggregate neighbor features directly in the graph domain (message passing).
- GraphSAGE, GAT, MPNN — efficient, flexible, and generalize to unseen graphs.
- **Dominant choice for large-scale and inductive settings — including wireless networks.**

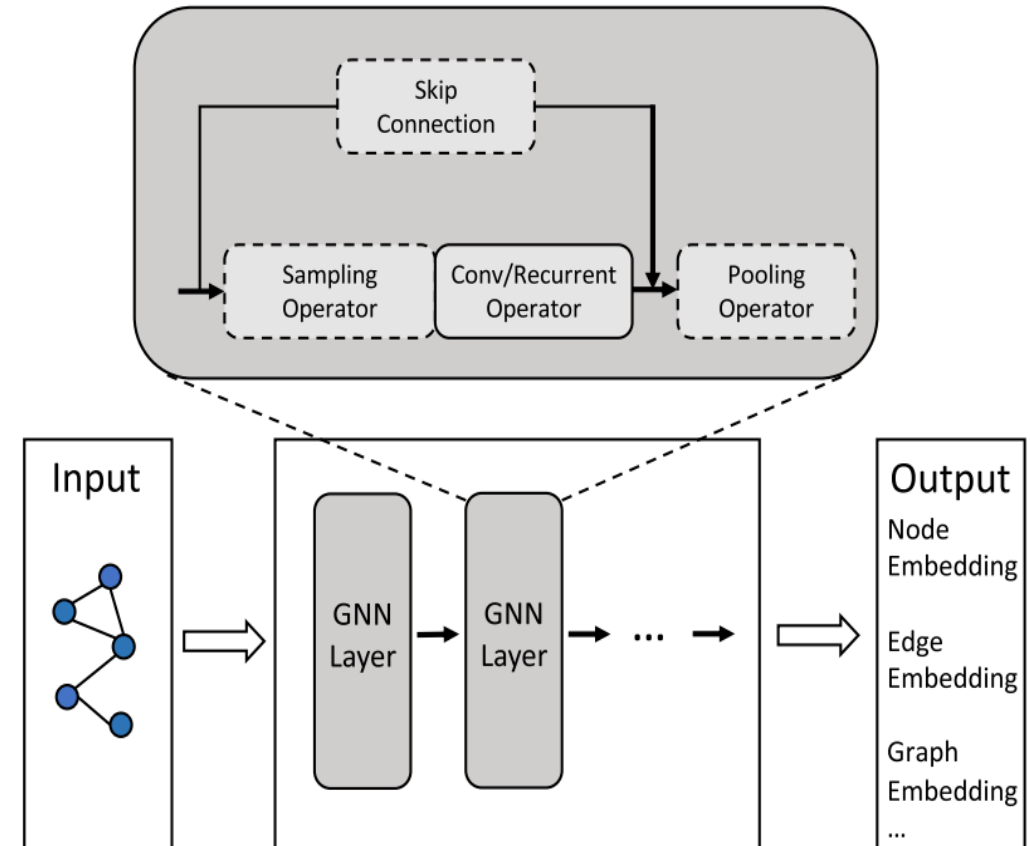
Zhou et al., “Graph Neural Networks: A Review of Methods and Applications,” AI Open, 2020.

One GNN layer, written out

Aggregation: $a_v^{(t)} = \text{AGGREGATE}(\mathfrak{t}) \left(\{h_u^{(t-1)} : u \in N(v)\} \right)$

Combination: $h_v^{(t)} = \text{COMBINE}(\mathfrak{t}) \left(h_v^{(t-1)}, a_v^{(t)} \right)$

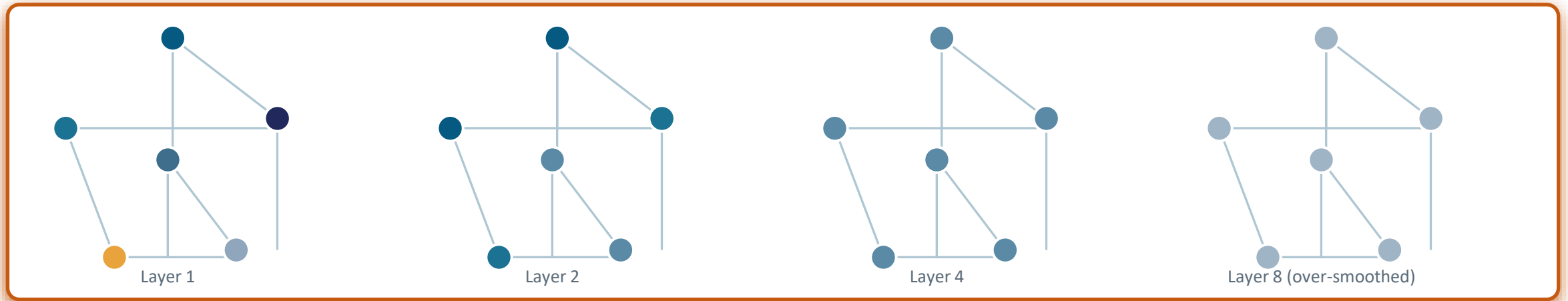
- AGGREGATE is permutation-invariant (sum, mean, max, or learned attention) so node order never matters.
- COMBINE and AGGREGATE are typically small MLPs whose weights are shared across every node in the graph.
- A final READOUT pools all node states into a graph-level vector, if the task is graph-focused.
- **This general layer is what will later be specialized into GNN architectures for wireless resource-management problems.**



Zhou et al., "Graph Neural Networks: A Review of Methods and Applications," AI Open, 2020; Wu et al., "A Comprehensive Survey on Graph Neural Networks," 2019.

Challenges

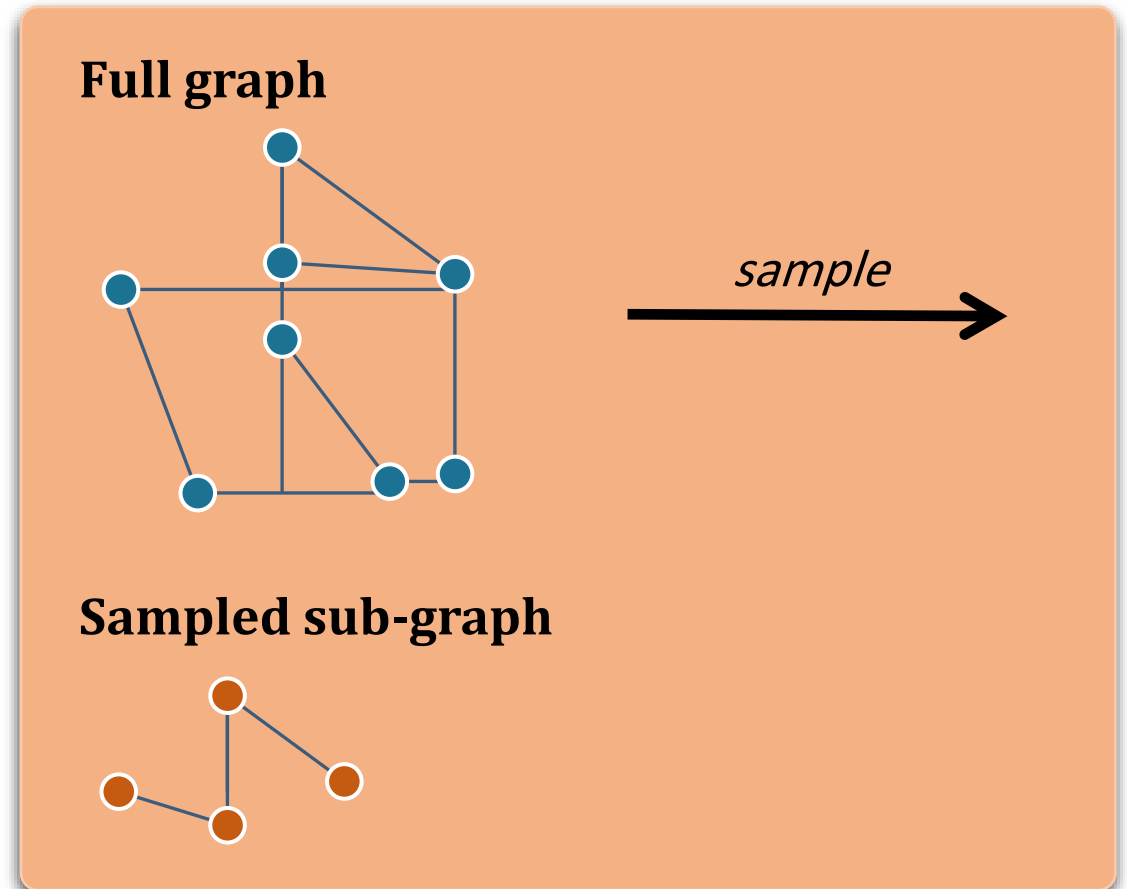
Limitation 1: over-smoothing with depth



- Each extra layer averages a node's state with more of the graph — after enough layers, all node representations converge to the same value.
- This caps how deep (and therefore how far-reaching) a GNN can practically be, limiting long-range dependency modeling.
- Mitigations: skip/residual connections (JKNet, ResGCN, DenseGCN), normalization, or careful choice of pooling.

Limitation 2: the scalability / completeness trade-off

- Full-batch training over a graph's whole adjacency matrix ($O(N^2)$ space) is infeasible for graphs with millions of nodes.
- **Sampling is the fix — but it always discards part of the graph:**
 - Node sampling (GraphSAGE): fixed number of neighbors per node.
 - Layer sampling (FastGCN): fixed set of nodes per layer.
 - Subgraph sampling (Cluster-GCN, GraphSAINT): train on sampled sub-graphs.
- **Every scheme trades some structural completeness for tractable memory and compute.**



Limitations 3 & 4 & 5: heterogeneity, dynamicity, robustness

Heterogeneous graphs

Most GNNs assume one node/edge type. Mixed types (users, APs, antennas) need custom meta-path or relation-specific weights.

Dynamic graphs

Topology and features can change over time; standard GNNs assume a static snapshot unless paired with RNNs/temporal attention.

Robustness & interpretability

GNNs are vulnerable to adversarial structure/feature perturbations and remain largely black-box models.

The deepest limitation: theory rarely guides design

- Empirically, GNNs beat MLPs/CNNs on graph tasks — but until recently, the only “proof” offered was the permutation-invariance argument.
- That argument explains why GNNs are not worse, but not how much better they are, nor which GNN design is best for a given problem.
- Without a quantitative theory, choosing an architecture is trial-and-error — costly for domains like wireless networks with strict latency and hardware budgets.

Preview of the theory to come

$O(n)$

× lower generalization error than MLPs

$O(n^2)$

× fewer training samples needed, for an n -node graph

Article

Graph Neural Networks for Wireless Communications: From Theory to Practice

Off-the-shelf MLPs/CNNs break down at scale

18%

Performance gap of a CNN-based beamforming model going from a 2-user to a 10-user D2D network.

>50%

Degradation of an MLP-augmented branch-and-bound access-point selector when user count/SNR shift slightly from training.

Why this matters for 5G/6G

- Densely deployed access points and hundreds of clients are the norm, not the exception.
- Client counts and SNR change dynamically — a model tuned for one snapshot must still work on the next.
- **⇒ Poor scalability + poor generalization make plain MLP/CNN architectures ineffective for practical deployment.**

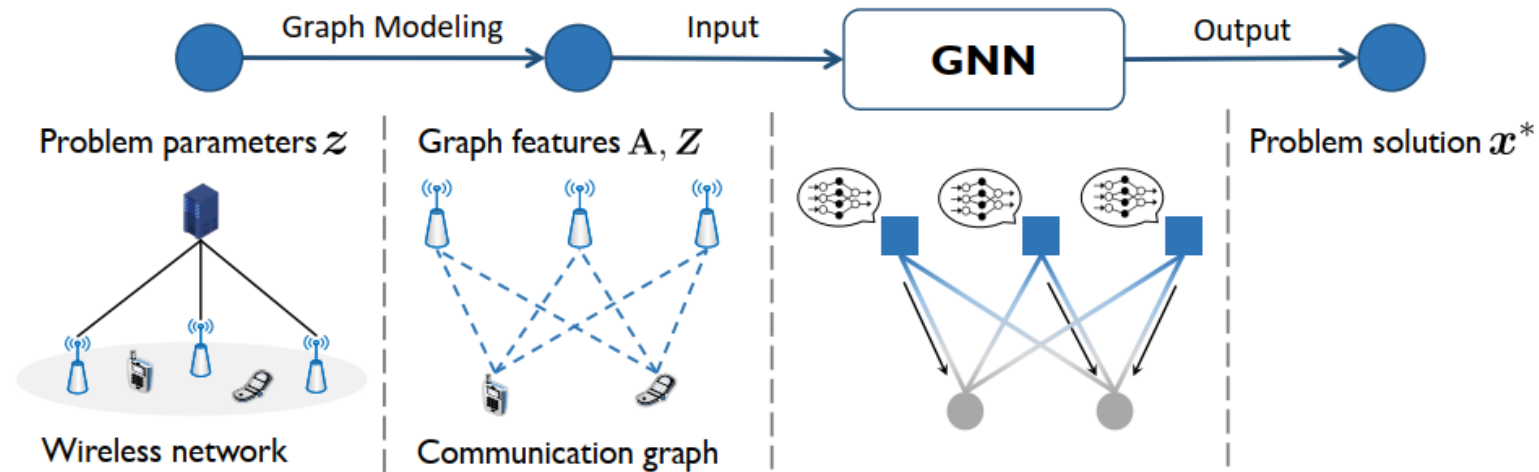
Yifei Shen *et al.*, "Graph Neural Networks for Wireless Communications: From Theory to Practice," in *IEEE Transactions on Wireless Communications*, May 2023.

Four contributions of the paper

- 1 Graph modeling framework**
A general recipe for casting wireless problems (power control, hybrid precoding, cell-free MIMO) as graph optimization problems.
- 2 GNNs $\equiv$ distributed message passing**
GNNs are shown to be the most powerful class of Distributed Message Passing (DMP) algorithms — they can represent WMMSE, Riemannian gradient, belief propagation, and more.
- 3 First generalization analysis**
Via PAC-learning + algorithmic alignment: GNNs need $O(n^2)$ fewer samples and reach $O(n)$ lower generalization error than MLPs, for an n -node graph.
- 4 Theory-guided design methodology**
Four concrete guidelines to improve any base GNN architecture, validated across three wireless problems.

Yifei Shen *et al.*, "Graph Neural Networks for Wireless Communications: From Theory to Practice," in *IEEE Transactions on Wireless Communications*, May 2023.

From wireless network to GNN solution



Two parallel threads :

- algorithm design — model as a graph, build a matching GNN, refine with theory-guided rules;
- theoretical analysis — prove GNNs can learn the optimal algorithm, then bound their sample complexity vs. MLPs.

Yifei Shen *et al.*, "Graph Neural Networks for Wireless Communications: From Theory to Practice," in *IEEE Transactions on Wireless Communications*, May 2023.

Wireless problems as graph optimization

$$\min_x f(\mathbf{X}, \mathbf{Z}, \mathbf{A}) \quad \text{subject to} \quad \mathbf{Q}(\mathbf{X}, \mathbf{Z}, \mathbf{A}) \leq 0$$

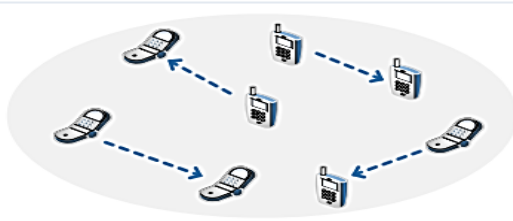
$\mathbf{X}$: optimization variable per node (e.g., transmit power) $\mathbf{Z}$: node feature matrix $\mathbf{A}$: adjacency feature tensor (edges)

- Communication graph $\mathbf{G}(\mathbf{V}, \mathbf{E}, \mathbf{s}, \mathbf{t})$: $\mathbf{s}$ maps a node to its feature vector, $\mathbf{t}$ maps an edge to its feature vector.
- Node feature matrix $\mathbf{Z} \in \mathbb{C}^{|v| \times d_1}$; adjacency feature tensor $\mathbf{A} \in \mathbb{C}^{|v| \times |v| \times d_2}$.
- This single formulation captures resource management, signal processing, and (via factor graphs) statistical inference problems in wireless networks.

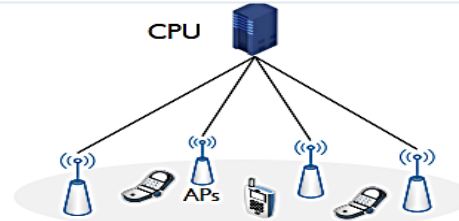
Yifei Shen *et al.*, "Graph Neural Networks for Wireless Communications: From Theory to Practice," in *IEEE Transactions on Wireless Communications*, May 2023.

Table I - Three running examples used throughout the paper

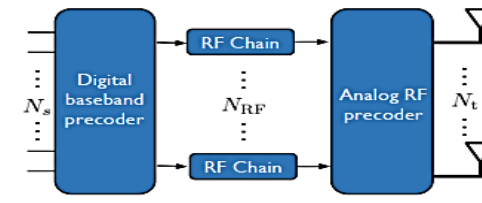
Problem Type	System Model	Graph Type	Node / Node Features	Edge / Edge Features
Resource management	D2D networks	Weighted graph	Transceiver pair / Direct-link channels	Interference links / Interference channels
Resource management	Cell-free networks	Bipartite graph	UEs & APs / No feature	Direct links / Large-scale coefficients
Signal processing	Hybrid precoding	Bipartite graph	Symbols & antennas / No feature	Connectivity / Digital beamforming matrix



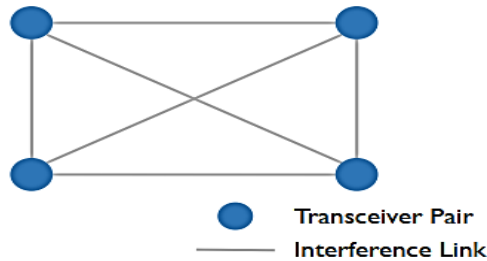
(a) D2D networks.



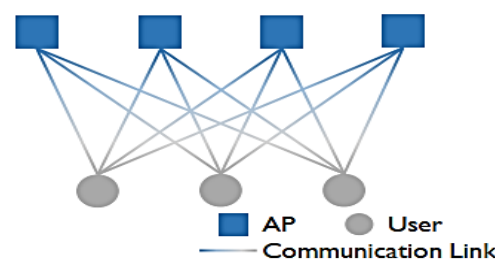
(b) Cell-free networks.



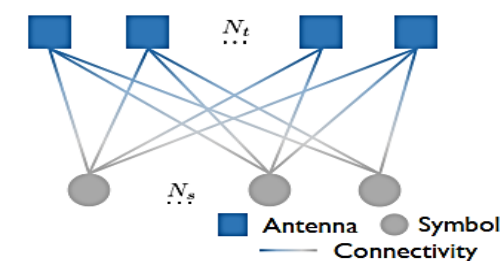
(c) Hybrid precoding.



(d) Graph modeling of D2D networks.



(e) Graph modeling of cell-free networks.



(f) Graph modeling of hybrid precoding.

Yifei Shen *et al.*, "Graph Neural Networks for Wireless Communications: From Theory to Practice," in *IEEE Transactions on Wireless Communications*, May 2023.

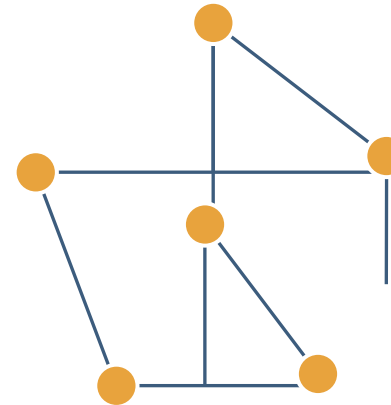
Aggregation and combination on a wireless graph

$$a_k^{(t)} = \text{POOL}_{j \in N(k)} \left(q_1^{(t)} \left(d_k^{(t-1)}, d_j^{(t-1)}, A_{(j,k,:)} \mid W_1^{(t)} \right) \right)$$

$$d_k^{(t)} = q_2^{(t)} \left(d_k^{(t-1)}, Z_{(k,:)}, a_k^{(t)} \mid W_2^{(t)} \right)$$

- ❑ Node k aggregates neighbors' previous states + edge features with a pooling function (sum/max/attention).
- ❑ It then combines the aggregate with its own feature Z_k and prior state to update d_k .
- ❑ After T layers, an output MLP maps $d_k^{(T)}$ to the decision variable, e.g. transmit power p_k .

Example: ECGNN for D2D power control



- ❑ Weighted graph of transceiver pairs; edge = interference link.
- ❑ $a_k = \sum_{j \neq k} \sigma \left(W_1^{(t)} d_k^{(t-1)} + W_2^{(t)} h_{j,k} + W_3^{(t)} d_j^{(t-1)} \right)$
- ❑ $p_k = \text{MLP} \left(d_k^{(T)} \right)$ — outputs the transmit power directly.

Yifei Shen *et al.*, "Graph Neural Networks for Wireless Communications: From Theory to Practice," in *IEEE Transactions on Wireless Communications*, May 2023.

Distributed Message Passing (DMP) algorithms

- A DMP algorithm: each node sends messages to neighbors, aggregates what it receives, and updates its own state — repeated for T communication rounds.
- This classic family unifies both optimization-based algorithms (WMMSE, Riemannian gradient, belief propagation) and GNN-based algorithms under one umbrella.
- **Theorem 1: for any graph optimization problem of the paper's form, there exists a DMP algorithm that solves it — because any such solution map is provably permutation-invariant.**

Algorithm 1: DMP round

- 1 Receive messages $m_{j \rightarrow i}$ from neighbors $j \in N(i)$
- 2 Aggregate: $y_i = g_1(m_{j \rightarrow i})$
- 3 Update state: $x_i = g_2(Z_i, x_i, y_i)$

Repeated for T rounds $\rightarrow$ output $[x_i^{(T)}]$

Theorem 2: GNNs can represent any DMP algorithm

WMMSE (Shi et al. 2011)

Classic optimization algorithm

Proposition 2: shown to be a DMP algorithm with $\leq 2T$ iterations.

Riemannian gradient (Yu et al. 2016)

Hybrid precoding solver

Proposition 1: its update rule is rewritten exactly as a DMP message-passing step.

ECGNN (learned)

GNN for D2D power control

A GNN layer is itself already a DMP step — with weights learned from data instead of hand-derived.

Corollary : since (1) every graph-optimization problem has a solving DMP algorithm (Thm 1) and (2) GNNs can represent any DMP algorithm (Thm 2) — there exists a GNN that solves any such wireless problem.

Yifei Shen *et al.*, "Graph Neural Networks for Wireless Communications: From Theory to Practice," in *IEEE Transactions on Wireless Communications*, May 2023.

Why GNNs need far fewer samples: PAC-learning + algorithmic alignment

- ✓ PAC-learning asks: how many training samples N guarantee error $< \epsilon$ with probability $\geq 1-\delta$?
- ✓ Algorithmic alignment (Xu et al., 2020): a network “aligns” well with a target algorithm if it decomposes into modules that each learn a simple sub-function — better alignment $\Rightarrow$ lower sample complexity N .
- ✓ Lemma 1: a GNN aligns with a T -iteration DMP algorithm using $O(\frac{TC}{|V|})$ samples per module; an MLP needs $O(TC \cdot |V|)$ — a factor of $|V|^2$ more overall.

$O(n^2)$ times fewer training samples
than MLPs

$O(n)$ times lower generalization error
than MLPs (Thm 4)

Yifei Shen *et al.*, "Graph Neural Networks for Wireless Communications: From Theory to Practice," in *IEEE Transactions on Wireless Communications*, May 2023.

Four guidelines for enhancing a base GNN

1

Simplify message passing

Fewer / more targeted incoming messages reduce the complexity of the function each module must learn.

2

Pick a matching pooling function

Max for sparse influence (e.g., strongest interferer); sum for aggregate statistics; attention when correlations matter.

3

Improve aggregation & combination nets

Use unrolling from a known classical algorithm (e.g., WMMSE) instead of a generic 1-layer MLP for q_1, q_2 .

4

Add input embeddings

Lift low-dimensional edge/node features into a higher-dimensional space so the network does not ignore them.

Yifei Shen *et al.*, "Graph Neural Networks for Wireless Communications: From Theory to Practice," in *IEEE Transactions on Wireless Communications*, May 2023.

D2D Power Control — Sample Efficiency

Training samples needed to reach matching performance (log scale)



Headline result

For K=20 users, PCGNN trained with just 20–40 samples matches an MLP trained with 16,000–64,000 samples.

Versus #transceiver pairs (fixed 100k samples)

PCGNN and ECGNN performance stays stable as the network grows; MLP performance decays linearly. PCGNN even surpasses “Best FPlinQ” at large user counts.

Yifei Shen *et al.*, "Graph Neural Networks for Wireless Communications: From Theory to Practice," in *IEEE Transactions on Wireless Communications*, May 2023.

D2D Robustness to SNR & Channel Shift

Table II — Robustness to transmit-power / SNR level (K=10, normalized by Best FPlinQ)

Tx Power (dBm)	PCGNN	PCGNN (Full Training)	FPlinQ
0	96.12%	97.10%	93.43%
10	96.80%	97.20%	93.75%
20	97.00%	97.39%	93.62%
30	96.65%	97.12%	93.31%
40	96.90%	97.13%	93.18%

Table III — Robustness to channel distribution shift (K=10)

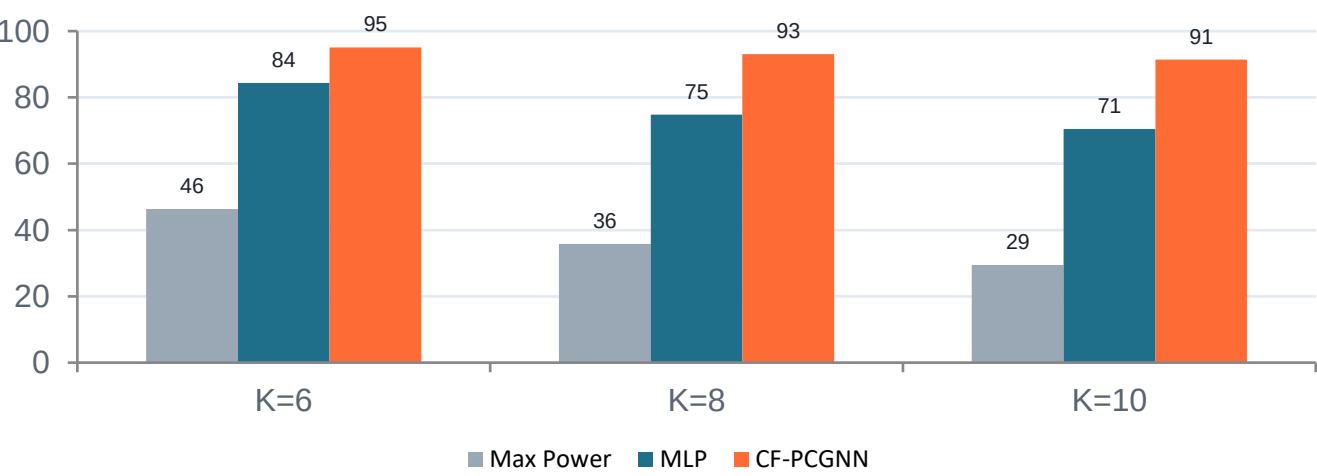
Setting	PCGNN	PCGNN (Full Training)	FPlinQ
User distribution shift	97.62%	97.78%	93.51%
Antenna height shift	97.63%	97.63%	93.66%
LoS → NLoS	94.43%	97.53%	93.51%
ITU → LTE	88.71%	96.57%	94.32%

Yifei Shen *et al.*, "Graph Neural Networks for Wireless Communications: From Theory to Practice," in *IEEE Transactions on Wireless Communications*, May 2023.

Cell-Free Massive MIMO Power Control

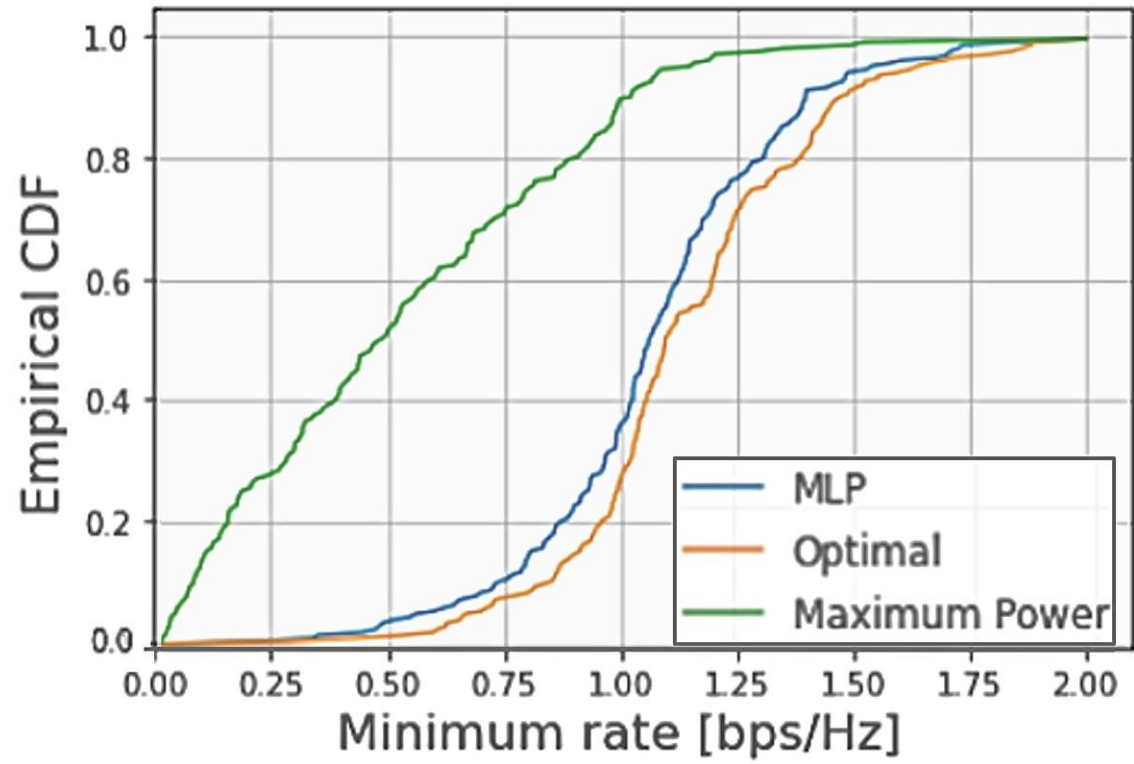
Table IV — Average minimum rate, normalized by the optimal algorithm (M = 30 APs)

K	Max Power	MLP	HetGNN	CF-PCGNN
6	46.3%	84.4%	94.9%	95.1%
8	35.8%	74.8%	91.9%	93.1%
10	29.4%	70.5%	90.8%	91.4%



CF-PCGNN also generalizes: trained on K=6 users, it keeps 90% + performance when tested on K=6–10 — and gains over MLP grow with K.

Yifei Shen *et al.*, "Graph Neural Networks for Wireless Communications: From Theory to Practice," in *IEEE Transactions on Wireless Communications*, May 2023.



Future work and Conclusion

Conclusion

- A unified framework for applying GNNs to wireless networks: graph modeling, architecture design, and theory, together
- First quantitative proof that GNNs need $O(n)$ times lower error and $O(n^2)$ times fewer samples than MLPs
- Extensive simulations confirm scalability, generalization, and large computational speedups over classic methods

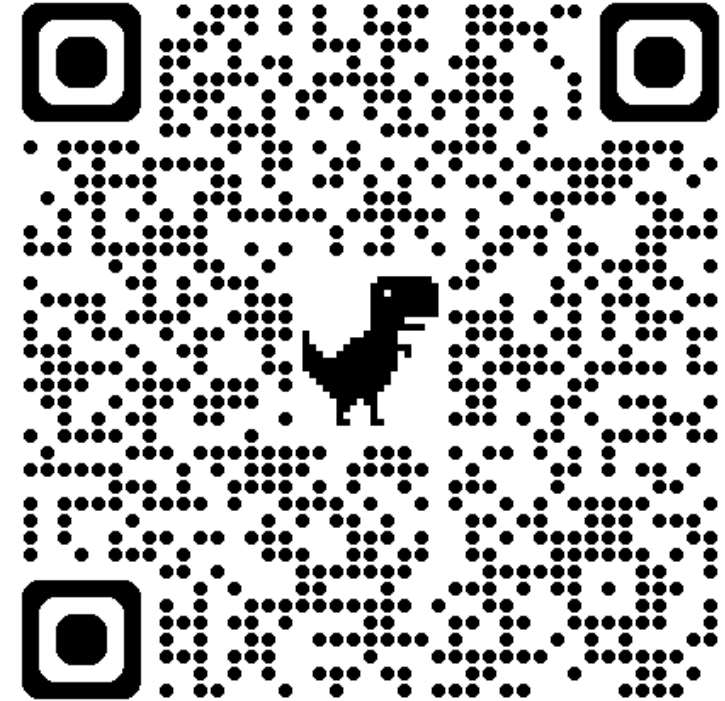
Future Directions

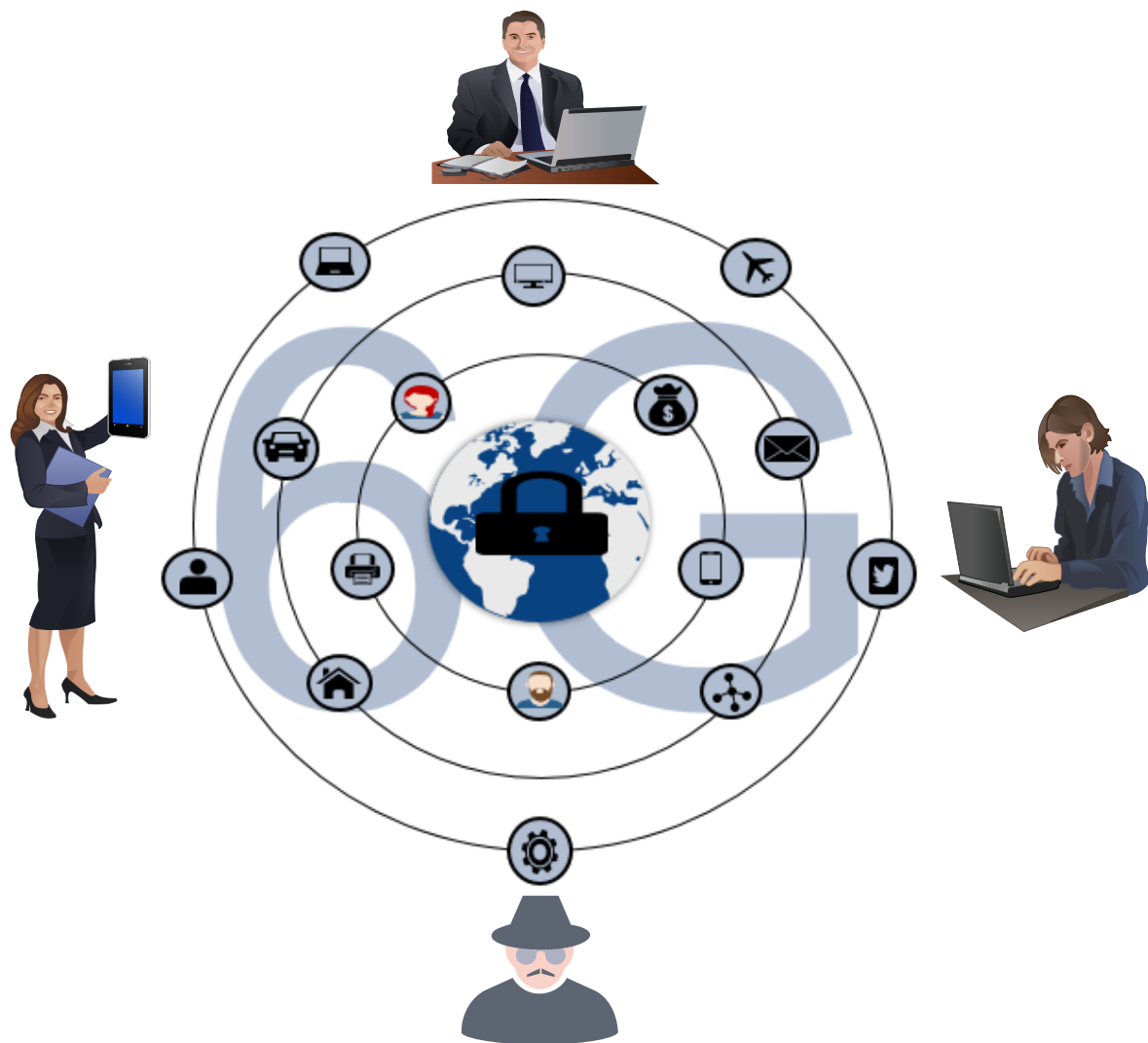
- Extend the graph-modeling & design framework to more wireless problems beyond D2D, cell-free, and hybrid precoding
- Combine GNNs with model-driven deep learning for tasks where reconstructing the underlying variable is easy
- Build a full theory of robustness to channel-distribution shift, beyond same-distribution generalization guarantees

- [1] F. Scarselli, M. Gori, A. C. Tsoi, M. Hagenbuchner, and G. Monfardini, "The graph neural network model," *IEEE Trans. Neural Netw.*, vol. 20, no. 1, pp. 61–80, Jan. 2009.
- [2] Z. Wu, S. Pan, F. Chen, G. Long, C. Zhang, and P. S. Yu, "A comprehensive survey on graph neural networks," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 32, no. 1, pp. 4–24, Jan. 2021.
- [3] J. Zhou, G. Cui, S. Hu, Z. Zhang, C. Yang, Z. Liu, L. Wang, C. Li, and M. Sun, "Graph neural networks: A review of methods and applications," *AI Open*, vol. 1, pp. 57–81, 2020.
- [4] J. Suárez-Varela, P. Almasan, M. Ferriol-Galmés, K. Rusek, F. Geyer, X. Cheng, X. Shi, S. Xiao, F. Scarselli, A. Cabellos-Aparicio, and P. Barlet-Ros, "Graph neural networks for communication networks: Context, use cases and opportunities," *IEEE Network*, vol. 37, no. 3, pp. 146–153, May/June 2023.
- [5] Y. Shen, J. Zhang, S. H. Song, and K. B. Letaief, "Graph neural networks for wireless communications: From theory to practice," *IEEE Trans. Wireless Commun.*, vol. 22, no. 5, pp. 3554–3569, May 2023.

Quiz

<https://forms.gle/hB2WokMPTqsVXi1L9>





Thank you for your
attention